HARAMAYA UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

INTERNSHIP REPORT

ON

DEVELOPMENT OF THE AMU DEDICATED STORE & RESOURCE MANAGEMENT SYSTEM AT ARBA MINCH UNIVERSITY ICT DIRECTORATE

An Internship Report Submitted to the Department of Information Technology, College of Computing and Informatics, Haramaya University, in Partial Fulfillment of the Requirements for the Internship/Practical Attachment Programme

Prepared By:	Yonas Girma
Student ID No.:	2749/16
Department Advisor:	Mr. Taddese A.
Hosting Organization:	Arba Minch University (AMU), ICT Directorate (ICTD)
Immediate Supervisor:	Mr. Chirotaw Kentib, Application Development Team Leader, ICTD
Internship Duration:	July 2 to September 2, 2026 (45 Working Days)
Submission Date:	

Haramaya, Ethiopia

September 2026

DECLARATION

I, Yonas Girma, bearing ID No. 2749/16, hereby declare that this internship report, entitled "Development of the AMU Dedicated Store & Resource Management System," is my own original work. It is based on my practical attachment at Arba Minch University ICT Directorate and has not been submitted, in whole or in part, for the award of any degree, diploma, or certificate at this or any other institution. All sources of information used in the preparation of this report have been duly acknowledged.

Advisor's Approval

This internship report has been submitted for examination with my approval as the university academic advisor.

Name: Mr. Taddese A.

Signature: _____

Date: _____

Executive Summary

This report details my 45-working-day practical attachment (July 2 to September 2, 2026) at the Arba Minch University ICT Directorate (ICTD), completed in partial fulfillment of the requirements for the Department of Information Technology at Haramaya University.

Attached to the Application Development team, our intern team set out to replace AMU's manual, paper-based requisition workflow with a custom web application: the AMU Dedicated Store & Resource Management System. Focusing on the internship-period baseline (evaluated around commit 01b0e84), we utilized NestJS, PostgreSQL (Prisma ORM), React 18, and Docker Compose to build an auditable platform that tracks stock intake, departmental requests, storekeeper issuance, and manager approvals across university departments and dedicated stores.

The resulting baseline prototype is designed to reduce delays associated with physical paper routing through current, transaction-driven stock-balance tracking, role-based access control combined with scope-based authorization, and eight operational inventory and activity reports. Beyond full-stack development practice, the internship provided firsthand experience in working with database transactions and concurrency risks, coordinating work in Git, and translating institutional rules into working code.

Acknowledgement

I would like to express my sincere gratitude to the Department of Information Technology, College of Computing and Informatics, Haramaya University, for arranging this internship opportunity and for the academic guidance provided throughout the attachment period.

I am especially grateful to my department advisor, Mr. Taddese A., for his continuous guidance, feedback, and encouragement before and during the internship, and for his support in the preparation of this report.

My heartfelt thanks go to my immediate supervisor at the host organization, Mr. Chirotaw Kentib, Application Development Team Leader at Arba Minch University ICT Directorate, for his patient mentorship, for entrusting me with meaningful responsibilities on the AMU Dedicated Store & Resource Management System, and for creating a supportive learning environment throughout my stay at ICTD.

I also thank the staff and fellow developers of the ICTD Application Development Team for their collaboration, technical guidance, and willingness to answer my questions, as well as my family and friends for their constant encouragement and support.

List of Acronyms

Acronym	Meaning
AMU	Arba Minch University
ICTD	Information and Communication Technology Directorate
IT	Information Technology
ERP	Enterprise Resource Planning
RBAC	Role-Based Access Control
JWT	JSON Web Token; used for access and refresh tokens
API	Application Programming Interface
REST	Representational State Transfer
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
UI / UX	User Interface / User Experience
SPA	Single Page Application
DB	Database
SQL	Structured Query Language
CSV	Comma-Separated Values
PDF	Portable Document Format
E2E	End-to-End (Testing)
TXN	Transaction (ledger entry code)

List of Tables and Figures

Tables

- Table 1: Departments and Store Seeded in the Demonstration Data
- Table 2: Backend Technology Stack
- Table 3: Frontend Technology Stack
- Table 4: Core Backend Feature Modules
- Table 5: Demo User Roles and Responsibilities

- Table 6: Summary of Principal Use Cases

Figures

- Figure 1: System Architecture Diagram of the AMU Dedicated Store & Resource Management System
- Figure 2: Requisition Approval and Issuance Workflow
- Figure 3: Use Case Diagram for Requisition and Inventory Management
- Figure 4: Use Case Diagram for Access and User Administration
- Figure 5: Use Case Diagram for Oversight and Reporting

Table of Contents

Student's Declaration 2
Advisor's Approval 2
Executive Summary 3
Acknowledgement 4
List of Acronyms 5
List of Tables and Figures 6
Tables 6
Figures 6
Table of Contents 7
Chapter One: Introduction 1
1.1 Background of the Organization 1
1.2 Vision and Mission 1
1.3 Main Products or Services 1
1.4 Main Customers or End Users 2
1.5 Organizational Structure 2
1.6 Work Flows 3
Chapter Two: Overall Internship Experience and Specific Work 4
2.1 Reason for Selecting the Organization 4
2.2 Section Worked In and Reason 4
2.3 Work Flow in the Section 6
2.4 Informal Leadership Role Within the Team 6
2.5 Tasks and Work Pieces Executed 6
2.6 Technical Knowledge and Skills Applied from Coursework 7
2.7 Programming Languages, Tools, and Techniques Used 7
2.8 Major Challenges Faced 8
2.9 Measures Taken to Overcome the Challenges 8
Chapter Three: Selection, Design, and Execution of the Project 10
3.1 Project Title and Short Summary 10
3.2 Problem Statement and Justification 10
3.3 Objectives 10
3.4 Methodology 11
3.5 Literature Review 14
3.6 Results and Discussion 15
3.7 Recommendations Regarding the Identified Problems 16
Chapter Four: Benefits Gained from the Internship and Reflection 17
Chapter Five: Conclusion and Recommendations 19
5.1 Overall Conclusion 19
5.2 Recommendation Regarding the Hosting Company 19
5.3 Recommendations for Improving the Internship 19
References 20

Appendices 21
Appendix A: Core Backend Feature Modules 21
Appendix B: System Access Points (Development Environment) 21
Appendix C: Project Source Repository 21

Chapter One: Introduction

1.1 Background of the Organization

Arba Minch University (AMU) traces its origin to the Arba Minch Water Technology Institute, established in September 1986 to train professionals in water resources. The institute was transformed into a full university in June 2004 and has since grown into one of Ethiopia's national universities, located in Arba Minch town, South Ethiopia Regional State, approximately 435 kilometres south of Addis Ababa. AMU today comprises multiple colleges and schools, as well as institutes such as the Arba Minch Water Technology Institute (AWTI) and the AMU Institute of Technology (AMUIT), spread across several campuses.

The internship was hosted by the university's Information and Communication Technology Directorate (ICTD), the central unit responsible for AMU's network infrastructure, enterprise information systems, and technical support services. Within ICTD, the Application Development Team designs, builds, and maintains custom institutional software systems used across the university's colleges, departments, and administrative directorates.

1.2 Vision and Mission

AMU's stated vision is to become one of the leading and most competitive universities in Ethiopia and Africa, and a well-recognized institution globally in teaching-learning, research, and community service, by the year 2030.

The university's mission centers on offering relevant and quality education and training, conducting problem-solving and indigenous-knowledge-integrated research, and adapting and transferring skills and technologies that support socio-economic development, alongside delivering accessible community services. The ICTD supports this mission directly by digitizing and modernizing the university's internal administrative and operational processes, of which the Store & Resource Management System is one example.

1.3 Main Products or Services

At the university level, AMU's core offerings are undergraduate and postgraduate academic programs, applied and problem-solving research, and community and consultancy services across its colleges and institutes.

At the level of the hosting unit, ICTD's core services include: campus network and infrastructure management; design, development, and maintenance of enterprise software systems (such as student information systems and internal ERP-type platforms); technical helpdesk and support services for staff and students; and digital-transformation initiatives, of which the Store & Resource Management System developed during this internship is a current example.

1.4 Main Customers or End Users

The university's principal beneficiaries are its students, academic and administrative staff, government stakeholders, and the surrounding community. For the specific system built by ICTD during this internship, the end users are internal: academic and administrative staff who file material requisitions, store managers who govern the item catalog and approve requests, storekeepers who execute physical stock movements, internal auditors who review transaction and audit trails, and system administrators who manage accounts and configuration, spanning AMU's colleges, departments, and directorates.

1.5 Organizational Structure

AMU is led by a President supported by Vice Presidents for academic affairs, administration, and research and community service, under whom the university's colleges, institutes, schools, and central directorates, including ICTD, operate. Within ICTD, the Application Development Team, led by the internship supervisor, operates alongside teams responsible for network and infrastructure, systems administration, and technical support.

The system built during the internship models AMU’s institutional structure in a deliberately simplified form. Departments are stored as a flat list; each store may be linked to an owning department, and users, employees, and requisitions are each tied to a department. Every user also carries a scope (global, organizational, or store-level) that limits which stores they can manage. A multi-level organizational tree (university → college → department) was explored in early design iterations but was simplified in the final version. Table 1 lists the departments and the store seeded in the demonstration data.

Table 1: Departments and Store Seeded in the Demonstration Data

Department / Store	**Code**
AMU Central & ICT Resource Store (store, owned by the STORE department)	STORE-MAIN
Computer Science Department	CS
Electrical Engineering Department	EE
Store & Inventory Management	STORE
General Administration	ADMIN
Finance & Procurement	FIN

1.6 Work Flows

At the system level, the store workflow supported during the internship baseline has two primary paths. In the requisition path, a requester (academic staff) files a request for materials; the Store Manager reviews it and approves or rejects it; and, once it is approved, the Storekeeper issues the items from stock. In the direct-operations path, the Storekeeper records stock-in from suppliers, direct stock-out to employees or departments, returns, stock-count adjustments, and transfers. Every stock operation writes a typed transaction record and updates store stock balances, with key actions producing protected audit records and in-app notifications. Subsequent post-internship development expanded this workflow to incorporate purchase orders, inspected goods receipts, cycle-count sessions, structured return inspections, transfer lifecycle tracking, request reservations, partial request issuance, and request closure handover evidence.

Chapter Two: Overall Internship Experience and Specific Work

2.1 Reason for Selecting the Organization

The internship placement letter issued by the Department of Information Technology was open, which meant I was free to choose my hosting organization instead of being assigned one. I treated this as an important decision and set three criteria before choosing: the organization had to run real, production-scale IT infrastructure, it had to employ dedicated technical staff who could guide an intern, and it had to offer practical software engineering work that matched what I had studied.

Arba Minch University's ICT Directorate was the organization I was specifically interested in, because I believed it had the right resources and capability to meet all three criteria. ICTD is the central unit responsible for the network, enterprise systems, and technical support of a university with multiple colleges and campuses, so it works with technology at a genuine institutional scale. Its Application Development Team designs and builds software for the university, which meant I could expect to take part in real development work alongside professional developers rather than only observing. I saw this as the best setting for applying my coursework in database systems, software engineering, and web programming, and for learning practices that classrooms rarely cover, such as team workflows, deployment, and working to institutional requirements.

To test this belief, I also looked at Arba Minch Referral Hospital as an alternative, and it did not meet my criteria. It had no servers of its own, its Health Management Information System (HMIS) was configured and hosted centrally by AMU's own staff, and its IT presence consisted mainly of health-science and HMIS-trained students who had been taught to operate the system rather than a functioning IT department. In practice, the hospital's core network and system administration was managed remotely by AMU. The fact that AMU's staff were the ones running another institution's systems strengthened my conviction that the capability I was looking for was at ICTD, which offered real infrastructure, dedicated technical staff, and a hands-on software project to design and build.

2.2 Section Worked In and Reason

I joined ICTD's Application Development section as an application developer / software programmer. The project I worked on was a system to replace AMU's manual, paper-based

store and requisition management, and my interest in it was genuine. As I learned how departments and stores at AMU requested and tracked materials, the problem was easy to see: stock was recorded on paper, requests moved slowly between campuses, and every audit meant compiling records by hand. I was inspired by the idea that a well-designed system could remove these delays and give the university accurate, auditable stock information. Before committing to the project, I discussed it with my department advisor, Mr. Taddese A., and the discussion confirmed that it was a worthwhile choice, because building the system from the ground up would give me direct, end-to-end exposure to full-stack enterprise software development, covering requirement analysis, database design, backend and frontend implementation, testing, and deployment, and because it closely matched my coursework in software engineering, database systems, and web technologies. This was a real institutional problem rather than an add-on to an already-running production system, and I wanted to help solve it. I was also occasionally invited to sit in on sessions with the Networking team, which provided a basic theoretical and practical introduction to networking concepts outside the core assignment.

2.3 Work Flow in the Section

The intern team followed an Agile-inspired iterative development approach. Its practices were task backlog grooming and assignment, local feature development on isolated Git branches, peer code review, manual and automated testing, and production configuration with Docker Compose and an Nginx reverse proxy. Work moved forward in small increments, and each increment was built, reviewed, tested, and shown before the next one began.

This approach was the most suitable process for our situation, for several reasons. First, the internship had a fixed window of 45 working days, and an iterative process produces working, demonstrable software early and continuously instead of leaving everything to the end. Second, although the problem and the use-case diagrams were clear at a high level, the detailed design was refined as we built. Early design iterations explored much larger capabilities, such as a configurable multi-step approval workflow, procurement, and an asset registry, and working iteratively let the team narrow the scope to the core store workflow before time was lost (see Section 3.7). Third, a sequential Waterfall process would have required all analysis and design to be completed before any coding, with testing last, when mistakes are costliest to fix and no time would remain to correct them.

Fourth, a full Agile framework such as Scrum, with formal sprint ceremonies and defined roles, would have added overhead that a small student team on a short attachment could not justify, which is why we adopted only the lightweight practices that gave the most value. Finally, short cycles with feature branches and peer review suited a team whose members had different levels of experience: work could be divided safely, mistakes were caught in review before they reached the shared code, and regular check-ins with the supervisor against the agreed use-case diagrams kept every increment aligned with the real requirements.

2.4 Informal Leadership Role Within the Team

Although not the officially appointed lead, I took on an informal team-lead role among the student peers on the Application Development Team, since several teammates were less self-taught and had limited prior exposure to professional development practices. This included introducing teammates to Git and GitHub workflows (branching, committing, pull requests, and code review) and explaining a number of technical concepts they had not previously encountered. This was not without its own difficulty: it required patience in staying approachable and "on the same level" as teammates with less experience, and in explaining foundational concepts clearly without appearing condescending, while still keeping the team's overall pace and code quality on track.

2.5 Tasks and Work Pieces Executed

During the 45-day attachment, my core focus was the data layer, backend services, and key frontend requisition screens for the internship-period baseline (commit 01b0e84). Specifically, I:

- Designed and refined the PostgreSQL data schema using Prisma ORM, structuring models for departments, stores, users, employees, suppliers, materials and categories, stock summaries, requests, inventory transactions, notifications, and audit logs.

- Implemented NestJS controllers and services for inventory movement (Stock In, Stock Out, Returns, Adjustments, and Transfers) providing transactional protection for core inventory mutations and updating store stock balances.
- Built client-side requisition and approval screens using React 18, TypeScript, and TanStack Query, connecting them to backend REST endpoints.
- Configured multi-service Docker Compose setups to orchestrate the NestJS API, PostgreSQL, Redis, the Vite frontend, and an Nginx reverse proxy, with separate development and production configurations.
- Executed manual integration scenarios and worked with the automated Jest unit tests and Supertest end-to-end tests covering authentication, role and scope enforcement, and core API endpoints.
- Maintained the project's GitHub repository and drafted technical documentation and demonstration guides (README, system documentation, and a presentation guide) for system demonstrations.

2.6 Technical Knowledge and Skills Applied from Coursework

- Database Management Systems: relational modeling, normalization, and SQL, applied directly to Prisma schema design.
- Object-Oriented Programming and Software Engineering principles: applied through NestJS's modular, dependency-injected architecture.
- Web / Internet Programming: applied while building the React single-page application and REST APIs.
- Data Structures and Algorithms: applied when reasoning about efficient, real-time inventory aggregation logic.
- Computer Networks: applied in understanding client-server communication, HTTP, and reverse-proxy configuration.

2.7 Programming Languages, Tools, and Techniques Used

Tool / Technology	Role in the Project
TypeScript / JavaScript	Primary programming languages for backend and frontend
NestJS 10	Backend application framework (modular, dependency-injected)

Tool / Technology	Role in the Project
Prisma ORM 5	Type-safe database access layer and migrations
PostgreSQL 16	Primary relational database
React 18 + Vite	Frontend single-page application framework and build tool
Tailwind CSS	Utility-first frontend styling
TanStack Query	Server-state caching and data-fetching on the frontend
Zustand	Lightweight frontend state management
Docker & Docker Compose	Containerization and multi-service orchestration
Nginx	Reverse proxy for production deployment
JWT (JSON Web Tokens)	Stateless authentication (access + refresh tokens)
Swagger / OpenAPI	Interactive API documentation
Jest & Supertest	Unit and end-to-end backend testing
Git / GitHub	Version control and source hosting

2.8 Major Challenges Faced

- A steep learning curve adapting to the NestJS/Prisma ecosystem and TypeScript's strict typing, which went beyond what coursework had covered.
- Understanding the system’s authorization model, which combines five roles (Administrator, Store Manager, Storekeeper, Auditor, Requester) with global and store-level scope checks (with ORGANIZATION scope defined as a structural constraint rather than a universal filter across all APIs), and the rule that stock quantities change through controlled inventory and request services recording protected audit records.
- Keeping received, issued, and balance figures consistent: each stock movement and its stock-summary update are written together inside a Prisma database transaction, while concurrent stock-out requests remained a risk that needs stricter locking (see Section 3.7).
- Coordinating contributions within a shared Git repository without introducing merge conflicts or regressions.
- Limited prior exposure to containerized deployment and multi-service orchestration with Docker Compose.

- Informally guiding less-experienced teammates through Git/GitHub and other technical concepts while staying patient, approachable, and “on their level”, without slowing the team's overall progress.

2.9 Measures Taken to Overcome the Challenges

- Studying the official NestJS, Prisma, and React documentation alongside the project's internal architecture notes.
- Seeking guidance and code-review feedback from the supervisor and senior developers on the Application Development Team.
- Practicing incremental, test-driven changes and using the Swagger API documentation to verify endpoint behavior before wiring up the frontend.
- Pulling and rebasing frequently, and clearly communicating task ownership with teammates to avoid conflicts.
- Setting aside dedicated self-study time to learn Docker fundamentals through official documentation and hands-on experimentation.
- Deliberately slowing down to explain concepts step by step to less-experienced teammates, using patience and relatable examples rather than assuming shared background knowledge.

Chapter Three: Selection, Design, and Execution of the Project

3.1 Project Title and Short Summary

Project Title: AMU Dedicated Store & Resource Management System.

The project is a web-based inventory and resource-management platform developed for Arba Minch University as an ICTD-assigned internship challenge project to automate the registration, current, transaction-driven stock-balance tracking, departmental requisition, supplier receiving, and audit reporting of dedicated institutional stores. Assigned as a student challenge project, the team designed and implemented the baseline platform under the supervisor's guidance and periodic review.

3.2 Problem Statement and Justification

Prior to this system, store inventory and departmental requisition operations at AMU relied on decentralized, paper-based bookkeeping. This made tracking stock accurately challenging, slowed down request processing across campuses, and required substantial manual effort when compiling paper records for audits.

These problems justified the need for a centralized, web-based platform capable of enforcing institutional accountability and auditability, while still preserving the operational speed that individual departments require to acquire and transfer resources.

3.3 Objectives

General Objective: To design and build a web-based resource and store management platform that automates registration, movement tracking, requisition approval, and audit reporting for AMU's dedicated institutional stores.

Specific Objectives:

- Digitize item cataloging with real-time, dynamically computed stock balances using Prisma ORM 5.19.0.
- Implement a structured requisition → approval → issuance workflow with clear role-based accountability.
- Provide protected historical audit records designed to preserve operational activity logs for audit traceability.

- Generate eight operational inventory and activity reports required for university oversight, with browser-based CSV export and print-to-PDF functionality.
- Enforce an RBAC model across five distinct operational roles (Administrator, Store Manager, Storekeeper, Auditor, Requester) combined with scope-based authorization guards.
- Containerize the system for consistent, repeatable development and production configuration with Docker Compose and Nginx reverse proxy.

3.4 Methodology

The project followed an Agile-inspired iterative development approach, structured around requirements that were derived from the institutional problem description, use-case diagrams, and workflow analysis provided by the ICT Directorate supervisor. The team was explicitly directed to build strictly to those specified requirements and diagrams rather than expand scope on their own initiative. Requirement analysis was grounded in the university's actual store-operation pain points (see 3.2). System design followed a layered client-server architecture (Figure 1) in which departments and stores underpin a Material → Stock Summary → Inventory Transaction data model. Stock movements are recorded as typed ledger entries (stock in, stock out, return, adjustment, transfer) written together with the balance update inside a single database transaction, and requisition-driven issuance follows a request → manager approval → storekeeper issuance flow (Figure 2).

Implementation followed a modular, feature-based structure on the backend (see Table 4) and a component-based structure on the frontend. Testing combined backend unit tests with Jest and end-to-end API tests with Supertest. Deployment was containerized with Docker Compose across separate development and production configurations, fronted by an Nginx reverse proxy. Figure 1 shows the resulting architecture and Figure 2 the requisition workflow.

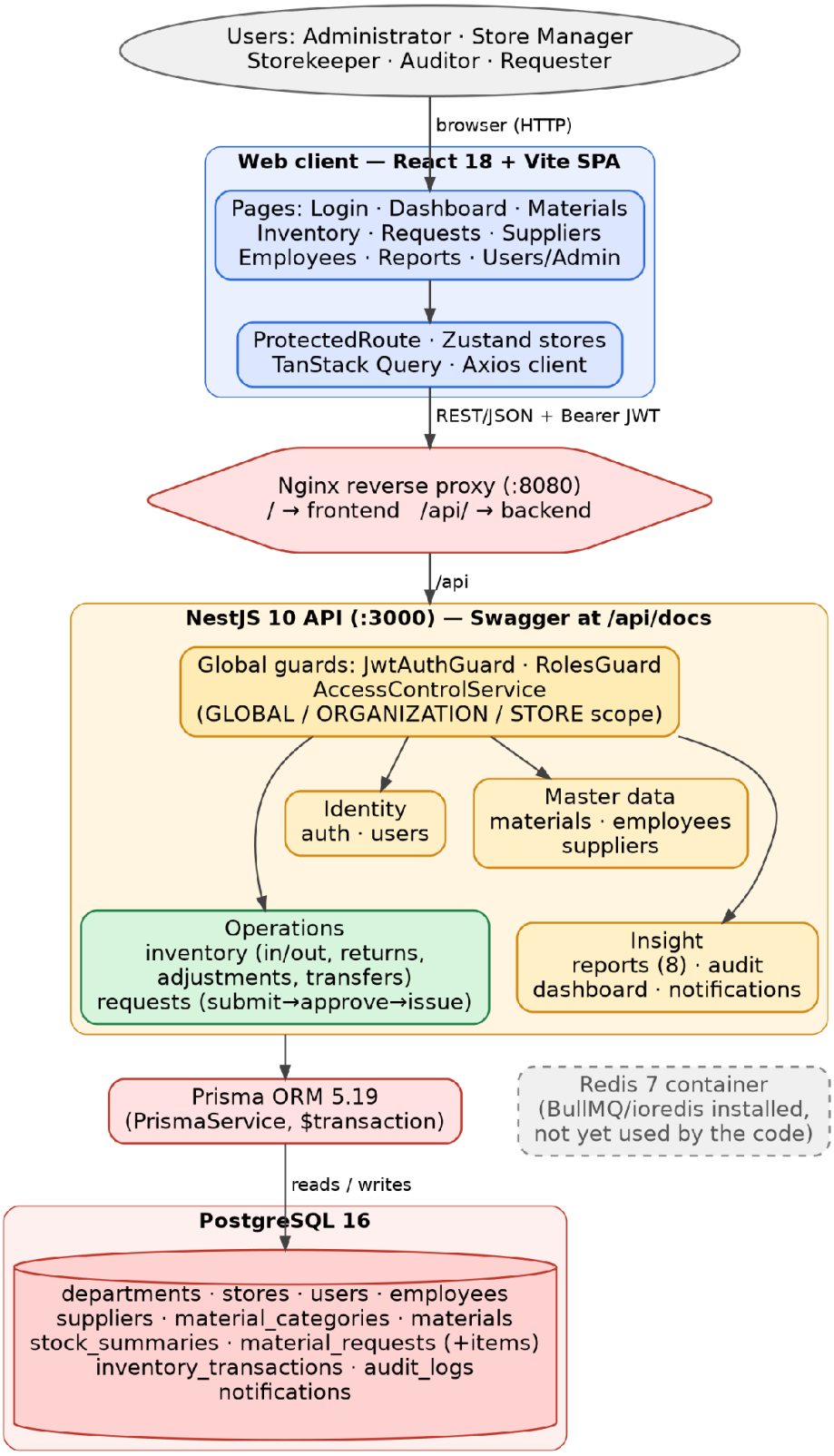


Figure 1: System Architecture Diagram of the AMU Dedicated Store & Resource Management System

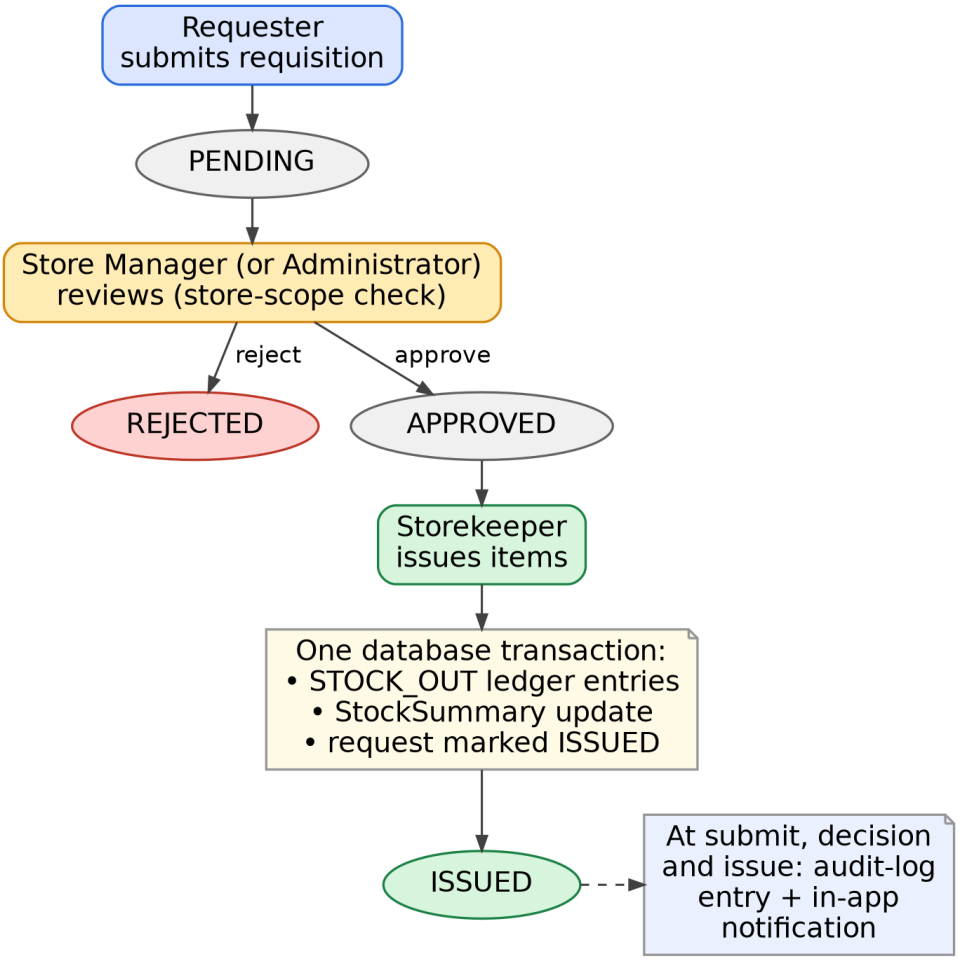


Figure 2: Requisition Approval and Issuance Workflow

Table 2: Backend Technology Stack

Technology / Tool	Version	Purpose
NestJS	^10.4.0	Core backend application framework
Prisma / @prisma/client	^5.19.0	ORM and database migrations
PostgreSQL	16	Relational database engine
Argon2	^0.40.3	Password hashing
@nestjs/jwt / passport-jwt	^10.2.0 / ^4.0.1	JWT authentication (access and refresh tokens)
@nestjs/swagger	^7.4.0	OpenAPI / Swagger documentation
BullMQ + ioredis	^5.12.0 / ^5.4.1	ioredis is utilized for IP rate limiting and brute-force account lockout; BullMQ is scaffolded but not fully implemented
Nodemailer	^6.9.15	Scaffolded for email alerts (not fully implemented in the active workflow)
class-validator / class-transformer	^0.14.1 / ^0.5.1	DTO validation

Table 3: Frontend Technology Stack

Technology	Version	Purpose
React	^18.3.1	UI library
Vite	^5.4.1	Frontend build tool / dev server
TypeScript	^5.5.4	Static typing
Tailwind CSS	^3.4.10	Utility-first styling
TanStack Query	^5.51.23	Server-state caching and data fetching
TanStack Table	^8.20.1	Data-table rendering
Zustand	^4.5.5	Client-side state management
React Hook Form + Zod	^7.52.2 / ^3.23.8	Form handling and schema validation
Recharts	^2.12.7	Dashboard charts
Axios	^1.7.4	HTTP client

Table 4: Core Backend Feature Modules

Module	Responsibility
auth	Registration, login, JWT access/refresh token issuance and session refresh, and current-session profile
users	User account management
employees	Employee and department directory
materials	Item / material catalog and categories
inventory	Stock in, stock out, returns, adjustments, transfers
requests	Requisition submission, approval, and issuance workflow
suppliers	Vendor directory and supplied-material tracking
reports	Eight operational inventory and activity reports; browser-based CSV export and print-to-PDF rendering
audit	Protected historical audit records capturing key system actions and user activity
dashboard	Aggregated operational dashboard data
notifications	In-app notifications (request events and low-stock alerts)

Table 5: Demo User Roles and Responsibilities

Role	Primary Responsibility
Store Manager	Material catalog governance, category creation, requisition review and approval

Role	Primary Responsibility
Storekeeper	Physical store operations: Stock In, Stock Out, Returns, Adjustments, Transfers, Issuance
Requester (Academic Staff)	Submits departmental material requisitions and tracks fulfillment status
Internal Auditor	Reviews transaction history and operational reports via the Reports & Analytics module (read-only)
Administrator (System Administrator)	User management, role assignment, department setup, protected audit log inspection, and JSON user record backup/restore

The functional requirements for the internship baseline were captured as a use case model. Figure 3 covers core store processes, Figure 4 covers account access and user administration, and Figure 5 covers oversight and reporting; Table 6 summarizes the principal use cases across the five roles (Administrator, Store Manager, Storekeeper, Auditor, Requester).

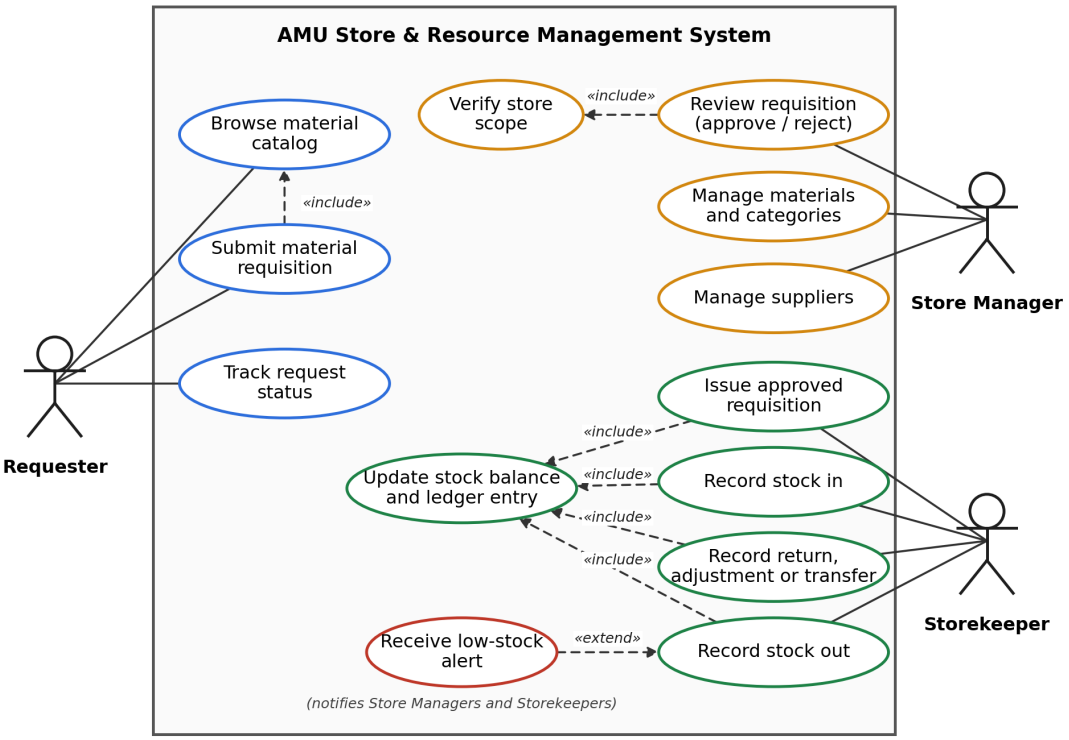


Figure 3: Use Case Diagram for Requisition and Inventory Management

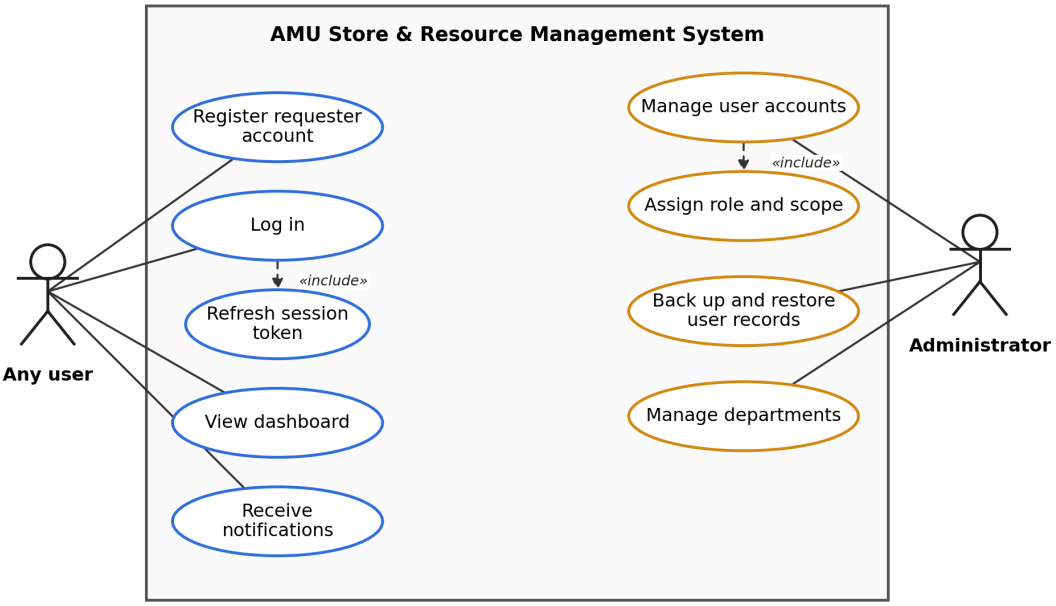


Figure 4: Use Case Diagram for Access and User Administration

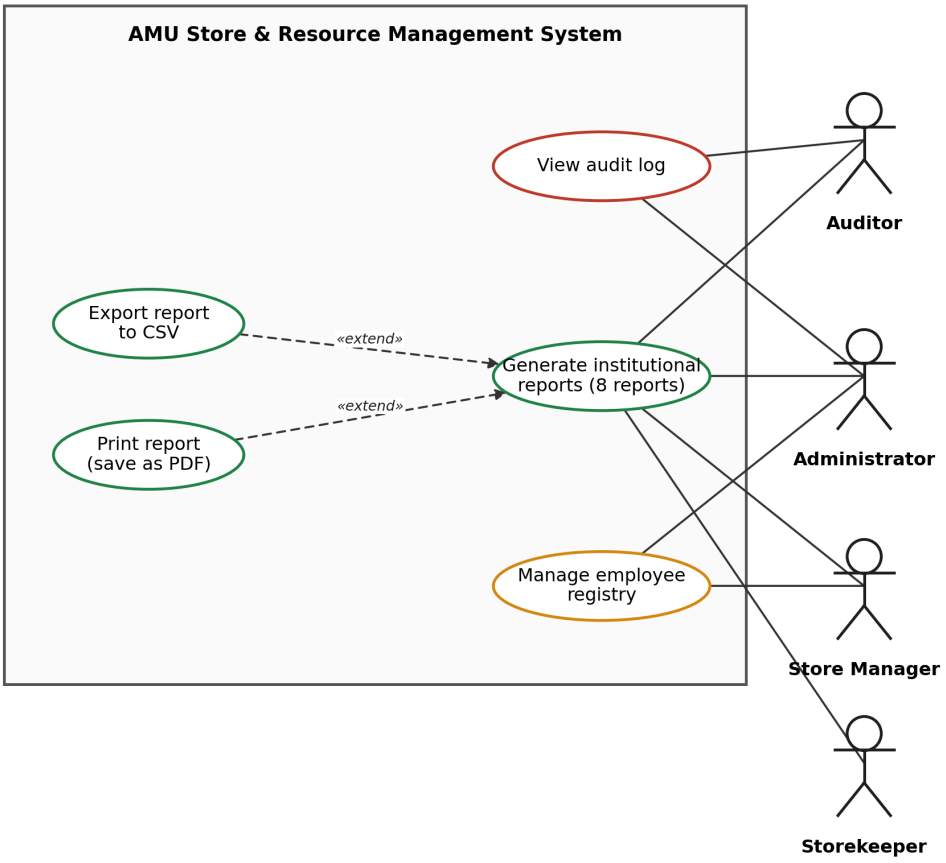


Figure 5: Use Case Diagram for Oversight and Reporting

Table 6: Summary of Principal Use Cases

Use case (actors)	Summary of the main flow
UC-01 Register and log in (Any user)	A visitor creates a Requester account from the login page, or signs in with e-mail and password. The API returns an access token and a refresh token, and the client renews the session automatically when the access token expires.
UC-02 Submit material requisition (Requester)	The requester browses the material catalog, selects materials and quantities, and submits the request. It is stored as PENDING and the Store Managers are notified.
UC-03 Review requisition (Store Manager, Administrator)	The reviewer opens a pending request; the system verifies that the reviewer's scope covers the store, and the request is approved or rejected. The requester is notified of the decision.
UC-04 Issue approved requisition (Storekeeper)	The storekeeper issues an APPROVED request. In one database transaction the system checks availability, writes STOCK_OUT ledger entries, decreases the stock summaries, and marks the request ISSUED.
UC-05 Record stock operations (Storekeeper, Store Manager, Administrator)	Stock in from a supplier, direct stock out, return, adjustment, or transfer is recorded as a typed ledger entry together with the stock-summary update. Managers and storekeepers are notified of stock-in and low-stock conditions.
UC-06 Manage materials, categories and suppliers (Store Manager; suppliers also Administrator)	Maintains the catalog (units, minimum-stock thresholds, barcode/QR identifier, categories) and the supplier records.
UC-07 Manage user accounts (Administrator)	Creates and edits users, assigns role and scope, activates or deactivates accounts, and exports or restores user records using a JSON backup.
UC-08 Manage departments and employees (Administrator; employees also Store Manager)	Maintains the department list and the employee registry used as the recipients of issued materials.
UC-09 View audit log (Administrator)	Reviews the protected historical audit records of user account changes, inventory stock movements, and requisition decisions.
UC-10 Generate operational reports (Administrator, Store Manager, Storekeeper, Auditor)	The user selects one of the eight operational inventory and activity reports, views paginated results, and exports to CSV or prints to PDF via the browser. **Eight Operational Inventory and Activity Reports:**

Use case (actors)	Summary of the main flow
	• Overall Inventory Status / Stock Balance Summary • Stock Receiving (Stock In) Ledger • Stock Issuance (Stock Out) Ledger • Departmental Requisition and Fulfillment Log • Low-Stock and Reorder Level Alert Report • Inter-Store and Inter-Departmental Transfer History • Supplier and Vendor Supply Summary • Audit Log and System Action History
UC-11 View dashboard and receive notifications (all users)	Shows summary indicators and in-app notifications such as new requests, decisions, issues, and low-stock alerts.

3.5 Literature Review

Existing research on inventory management systems generally agrees that traditional manual and spreadsheet-based record-keeping can be error-prone and inefficient, and many recent studies focus on replacing these methods with web- or mobile-based systems offering current, transaction-driven stock-balance tracking. AMU's store operations exhibited similar manual bottlenecks.

Literature on Enterprise Resource Planning (ERP) systems (e.g., Zhao & Tu, 2021) characterizes them as integrated platforms that unify departmental functions across an organization. Studies on higher education systems, such as a conceptual framework for ERP evaluation in Pakistani universities, note that research on ERP implementation and evaluation in higher education in developing countries remains limited, despite higher education institutions facing similar multi-departmental coordination challenges. This project applies established inventory-management principles, and a limited subset of ERP-style integration, to that context by addressing the operational needs of a decentralized Ethiopian public university.

Security literature on Role-Based Access Control (RBAC), such as the proposed NIST standard by Ferraiolo et al. (2001) and reviews of access control in information systems,

presents RBAC as a standard approach for managing permissions. Because roles alone may not restrict a user to a particular store or organizational unit, the system combines RBAC with explicit scope values (global, organizational, or store-level); in the current implementation the scope check is applied to store-management actions such as approving a requisition.

3.6 Results and Discussion

The internship-period baseline implementation (commit 01b0e84, evaluated around 8 October 2026) established core full-stack features: authentication with JWT access and refresh tokens, Redis-based login rate limiting and lockout; five operational roles (Administrator, Store Manager, Storekeeper, Auditor, Requester) with global and store-level access control; material cataloging, categories, departments, employees, suppliers, and stores; stock operations (stock in, stock out, returns, adjustments, transfers) providing transactional protection for core inventory mutations; a requisition workflow (PENDING, APPROVED, ISSUED, REJECTED); protected historical audit records; eight operational inventory and activity reports with browser-based CSV export and print-to-PDF; paginated reports and transaction history; and Docker Compose dev/prod setups with Nginx reverse proxy, tested via Jest and Supertest.

Demonstrated operational scenarios showed that a process which previously required manual paper reconciliation could be completed with transaction-driven stock balance updates and protected historical audit records, directly addressing the operational pain points described in Section 3.2. Unit prices recorded during stock movements serve as operational source-document fields rather than financial inventory valuation.

At the conclusion of the internship, the team formally presented and demonstrated the baseline system to the supervisor, who reviewed it against the original requirements and approved it. The source code remained in the student team's project repository.

The baseline was validated using seeded demonstration data and controlled test scenarios during the internship, as a student challenge project rather than a full university-wide production deployment. It should not be interpreted as a complete ERP, financial accounting system, or fixed-asset subledger.

3.8 Baseline Scope vs. Post-Internship System Evolution

To ensure full clarity and academic integrity, this report documents the internship-period baseline implementation demonstrated and evaluated around commit 01b0e84 on 8 October 2026. Subsequent post-internship repository development expanded the platform significantly beyond the 45-working-day attachment scope.

Later post-internship enhancements introduced purchase order workflows (draft, submit, approve, reject, cancel, partial-receipt); inspected goods receipts; cycle-count audit sessions; structured return inspections; full transfer dispatch/receipt lifecycle tracking; stock reservations; partial request issuance; request states (PARTIALLY_ISSUED, BACKORDERED, CANCELLED, CLOSED); request handover evidence and requester confirmation; transactional outbox processing; store-specific StockBalance models as authoritative store stock state; and additional inventory-control mechanisms. These post-internship additions reflect ongoing repository evolution and must not be interpreted as work completed during the 45-working-day internship period.

3.7 Recommendations Regarding the Identified Problems

- If ICTD wishes to move the project toward institutional use, it should establish a formal process for codebase handover and assign permanent maintainers within the Application Development section.
- Roll out a phased pilot within ICTD's own store before expanding the system to other colleges and departments.
- Provide structured training for storekeepers, requesters, and auditors ahead of any go-live.
- Establish a formal plan for migrating existing paper-based inventory records into the new system.
- Establish automated alerts for low-stock conditions and audit anomalies. Low-stock alerts appear as in-app notifications; while ioredis is utilized for IP rate limiting and brute-force account lockout on the auth service, Nodemailer and BullMQ remain scaffolded and can be fully integrated for scheduled and email alerts. User record backup/restore should be extended to scheduled full-database backups (e.g., pg_dump).

- Consider adding barcode or QR-code scanning at physical stock points, building on the item QR metadata already scaffolded in the system, to further reduce manual data-entry errors.
- Make direct stock-out operations safe under concurrent use by performing the balance check inside the database transaction (for example, with a conditional update or row-level locking), because the current stock-out path reads the balance before the transaction begins.
- Reintroduce, as later modules, the larger capabilities explored in early design iterations (a configurable multi-step approval workflow, procurement with purchase orders and goods receipts, multi-allocation distribution, an asset registry with borrowing and disposal, and a multi-level organizational hierarchy) only after the core store workflow has been adopted and proven.
- Evaluate replacing browser-based CSV and print-to-PDF exports with server-side Excel and PDF generation engines for heavy operational data, and ensure production credential rotation before live deployment.

Chapter Four: Benefits Gained from the Internship and Reflection

4.1 Technical and Practical Skills Learned

Working on a system modeled on a real university process helped me connect what I learned in class with actual software development. Putting relational database design and normalization into practice using Prisma ORM and PostgreSQL made database concepts much clearer than classroom exercises alone. I also learned a lot about concurrent inventory updates in NestJS: wrapping core stock movements in database transactions, and recognizing concurrency considerations. Setting up the application with Docker Compose gave me solid experience with containerization and managing multi-service setups.

4.2 Team Collaboration, Git Workflows, and Peer Mentorship

Since some of my student teammates were less familiar with professional tools, I helped guide our team's daily development work. I organized our workflow in Git and GitHub, showing teammates how to use branches, open pull requests, and resolve merge conflicts. This experience taught me patience and how to explain technical concepts clearly without slowing down our progress or creating frustration within the team.

4.3 Professional Ethics, Institutional Context, and Career Perspective

Working within the ICTD Application Development Team gave me a practical understanding of how software is built in a real organization. We focused on meeting the actual requirements rather than adding unnecessary features. Building software meant to handle university resources made me realize how critical data accuracy and proper audit logging are. Overall, the internship reinforced my goal of becoming a software engineer and showed me what it takes to work effectively on an engineering team.

Chapter Five: Conclusion and Recommendations

5.1 Overall Conclusion

The 45-working-day internship at Arba Minch University's ICT Directorate offered valuable practical experience in full-stack software development through the AMU Dedicated Store & Resource Management System project baseline. The experience helped connect classroom learning in databases, software engineering, and web development with real-world software design, helping prepare me for professional IT work.

5.2 Recommendation Regarding the Hosting Company

Arba Minch University's ICT Directorate is recommended as a strong host for future Information Technology student placements. It offers exposure to an industry-standard technology stack, a realistic, production-oriented project built around a real institutional process, and supervision from experienced developers willing to mentor students through code review and guided problem-solving.

5.3 Recommendations for Improving the Internship

- Provide a short, structured onboarding/orientation to the specific technology stack (NestJS, Prisma, React, Docker) at the start of the internship, rather than expecting interns to ramp up entirely on their own.
- Establish a clearer, written schedule of mentorship check-ins between the intern and supervisor.
- Define intern task scope more explicitly at the outset, with a short list of concrete deliverables for each phase of the internship.
- Grant interns access to a dedicated staging environment early, to reduce time lost to local environment setup.
- Encourage interns to maintain a running personal log of tasks and challenges throughout the internship, to make final report-writing easier and more accurate.

References

Arba Minch University. (2026). About AMU: History, vision and mission. Arba Minch University

Girma, Y. (2026). AMU Dedicated Store & Resource Management System [Source code repository]. GitHub. https://github.com/YonasGr/AMU-Resource-Management-System

NestJS Documentation. NestJS Docs

Prisma Documentation. Get started with Prisma | Prisma Documentation

React Documentation. React

Zhao, X., & Tu, C. (2021). Research and development of inventory management and human resource management in ERP. Wireless Communications and Mobile Computing, 2021, Article 3132062. Research and Development of Inventory Management and Human Resource Management in ERP - Zhao - 2021 - Wireless Communications and Mobile Computing - Wiley Online Library

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. ACM Transactions on Information and System Security, 4(3), 224-274.

A review of existing inventory management systems. (2024). ResearchGate. (PDF) A Review of Existing Inventory Management Systems

Inventory management concepts and implementations: A systematic review. (2022). ResearchGate. https://www.researchgate.net/publication/362344346

A conceptual framework for ERP evaluation in universities of Pakistan. (2014). arXiv:1402.0116. A Conceptual Framework for ERP Evaluation in Universities of Pakistan

Health information system role-based access control: Current security trends and challenges. PMC. Health Information System Role-Based Access Control Current Security Trends and Challenges - PMC

Appendices

Appendix A: Core Backend Feature Modules

audit · auth · dashboard · employees · inventory · materials · notifications · reports · requests · suppliers · users

Appendix B: System Access Points (Development Environment)

- Web Application Portal: http://localhost:5173 (or http://localhost:8080 in production)
- API Swagger Documentation: http://localhost:3000/api/docs
- Backend API (direct): http://localhost:3000

Appendix C: Project Source Repository

The complete source code for the AMU Dedicated Store & Resource Management System, developed by the intern team as their internship challenge project, is available at: https://github.com/YonasGr/AMU-Resource-Management-System

Repository state described in this report: branch main, commit 01b0e84 (8 October 2026). The repository history contains 27 commits total at HEAD from 27 July 2026 onward.